

Forecasting of Financial Time Series Using XGBoost and Transformer Models

Ramiro Javier Valdes Jara
Department of Industrial and Systems Engineering
University of Miami
Coral Gables, United States of America
rjv71@miami.edu

Abstract—This report presents a comparative study between gradient boosting (XGBoost) and transformer-based neural networks for multi-horizon forecasting and directional classification of financial time series.

I. INTRODUCTION

Financial time series forecasting is one of the most challenging tasks in applied machine learning due to the inherently noisy, non-stationary, and highly volatile nature of asset prices. Accurate short-term forecasting can improve portfolio allocation, risk management and automated trading strategies. In this context, two fundamental predictive goals are often pursued: estimating the future price level over a fixed horizon, and classifying the expected direction of movement (up or down) after a given number of timesteps.

Traditional machine learning models, such as gradient boosting trees, have demonstrated remarkable success in structured data problems. Among them, Extreme Gradient Boosting (XGBoost) has become one of the most powerful and efficient algorithms for tabular regression and classification tasks. Its ability to model nonlinear interactions make it a strong baseline for financial forecasting, especially when the input features are engineered from open-high-low-close-volume (OHLCV) data and related indicators.

On the other hand, recent advances in deep learning have introduced architectures capable of capturing long-range temporal dependencies in sequential data. The Transformer model, originally proposed for natural language processing, has been successfully adapted for time series forecasting due to its self-attention mechanism, which allows the model to weight the importance of past observations dynamically. Compared to traditional recurrent or convolutional architectures, the Transformer offers superior performance when dealing with multi-horizon predictions.

First, we aim to perform multi-step forecasting of asset prices, predicting the next h timesteps based on a historical input window of past observations. Second, we extend the same input representation to a directional classification task, where the model predicts whether the price after h timesteps will increase or decrease relative to the current value. By comparing XGBoost and Transformer architectures under both regression and classification settings, we seek to understand

the trade-offs between traditional machine learning methods and modern neural architectures for financial time series modeling.

II. DATA AND PREPROCESSING

A. Dataset Description

All experiments were conducted on high-frequency cryptocurrency market data, consisting of one-minute OHLCV (*Open, High, Low, Close, Volume*) records collected from Binance public archives for several trading pairs (e.g., BTC/USDT). Each record provides a comprehensive snapshot of price dynamics and market liquidity over time, making it a representative benchmark for financial time series modeling.

The data spans several weeks of continuous trading and was divided chronologically into training, validation, and test subsets with ratios of 70%, 15%, and 15%, respectively. This sequential split ensures that the forecasting and classification models are evaluated on unseen future data.

B. Feature Engineering and Modalities

To evaluate the influence of input richness and temporal representation, for each of the tasks (price forecasting or trend classification) five data modalities were constructed based on combinations of raw OHLCV features and engineered indicators.

a) (1) *XGBoost (Close-only)*.: The first modality uses only the closing price series. The model receives sliding windows of past close values of length L_{in} , flattened into tabular vectors suitable for tree-based regression.

b) (2) *XGBoost (OHLCV)*.: The second modality extends the input to include the full OHLCV structure. Each window concatenates open, high, low, close, and volume observations over the last L_{in} timesteps, forming an input of size $L_{in} \times 5$ per sample. The model predicts the next h closing prices using a direct multi-output regression setup.

c) (3) *XGBoost (OHLCV + Engineered Features)*.: To further enrich the input space, a third modality introduces a set of handcrafted, leak-safe technical indicators. These include logarithmic returns, the high–low range, rolling volatility of returns, standardized volume (z-score) and the Relative Strength Index (RSI-14). The augmented feature set therefore comprises eleven variables: the five OHLCV components plus

six engineered features. This representation aims to provide XGBoost with stationary and non-linear predictors without introducing temporal leakage.

d) (4) Transformer (Close-only).: The univariate Transformer model processes normalized sequences of closing prices. Input sequences of length L_{in} are normalized via z-scoring computed exclusively on the training split. Window extraction is performed, producing batched tensors of shape $[N, L_{\text{in}}, 1]$ for the encoder input and $[N, h]$ for the regression targets.

e) (5) Transformer (OHLCV).: The fifth modality generalizes the Transformer to multichannel input by providing all five OHLCV variables as separate feature dimensions. The preprocessing pipeline applies per-channel normalization, ensuring that each feature contributes proportionally to the attention mechanism. Multi-step windows are generated, yielding tensors of shape $[N, L_{\text{in}}, 5]$. The model forecasts the next h normalized close values, later denormalized for metric computation and visualization.

C. Task Design

Both regression and classification objectives share the same temporal setup. For forecasting, the model directly predicts the next h closing prices as

$$\hat{y}_{t+1:t+h} = f(x_{t-L_{\text{in}}:t}),$$

where $f(\cdot)$ denotes either the XGBoost or Transformer model. Evaluation is conducted across all horizons to measure error propagation as the prediction step increases.

For directional classification, the task is converted into a binary prediction of market movement after h timesteps:

$$y_t = \begin{cases} 1, & \text{if } p_{t+h} > p_t, \\ 0, & \text{otherwise.} \end{cases}$$

By aligning the task and windowing strategies across modalities, we ensure a fair comparative analysis of model architectures and input representations.

III. METHODOLOGY

A. XGBoost Model

The model is based on Extreme Gradient Boosting (XGBoost), an ensemble of decision trees that iteratively minimizes a differentiable loss function using second-order gradient statistics. XGBoost is a strong baseline for time series forecasting when the input sequence is represented as a flattened feature vector.

Each input sample corresponds to a temporal window of length L_{in} , either consisting solely of past closing prices or concatenated OHLCV features. When additional engineered indicators are enabled, the total input dimensionality expands to $L_{\text{in}} \times F$, where F denotes the number of input features (5 for OHLCV or 11 when feature engineering is applied). This vectorized representation transforms the sequential forecasting problem into a standard supervised regression task.

For the forecasting experiments, each model performs direct multi-output regression to predict the next h closing prices simultaneously:

$$\hat{\mathbf{y}}_{t+1:t+h} = f_{\text{XGB}}(\mathbf{x}_{t-L_{\text{in}}:t}),$$

where $f_{\text{XGB}}(\cdot)$ denotes the ensemble of regression trees. The objective function is the squared error loss,

$$\mathcal{L}_{\text{reg}} = \frac{1}{h} \sum_{i=1}^h (y_{t+i} - \hat{y}_{t+i})^2,$$

and optimization proceeds using additive tree boosting with shrinkage and column subsampling. The implementation employs the fast histogram grower, which reduces training time for large windowed datasets.

For directional classification, the output layer is adapted to a logistic regression objective. In this configuration, the model outputs a probability $\hat{p}_t = P(p_{t+h} > p_t)$, and predictions are thresholded at 0.5 to determine the up or down movement.

The hyperparameters were selected empirically to balance model complexity and generalization. The key parameters include a learning rate $\eta \in [0.05, 0.25]$, maximum tree depth between 6 and 9, and an ensemble size of 400–800 estimators. Early stopping on the validation loss is applied.

B. Transformer Model

The second model is based on the Transformer architecture, designed to capture temporal dependencies via attention mechanisms rather than fixed receptive fields or recurrences. The model follows an encoder-only configuration implemented in PyTorch, optimized for univariate and multivariate time series forecasting.

Each input sequence $\mathbf{X}_t \in \mathbb{R}^{L_{\text{in}} \times C}$, where C represents the number of input channels (1 for closing price or 5 for OHLCV), is first projected into a latent representation of dimension d_{model} through a linear embedding layer:

$$\mathbf{Z}_t = \text{Linear}(\mathbf{X}_t).$$

To retain temporal order information, a learnable positional encoding $\mathbf{P} \in \mathbb{R}^{L_{\text{in}} \times d_{\text{model}}}$ is added to the embedded sequence, yielding $\tilde{\mathbf{Z}}_t = \mathbf{Z}_t + \mathbf{P}$. The sequence is then processed by a stack of L Transformer encoder layers, each composed of multi-head self-attention and position-wise feed-forward sublayers:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax} \left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} \right) \mathbf{V},$$

where $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ are linear projections of $\tilde{\mathbf{Z}}_t$. The attention heads enable the model to dynamically focus on relevant past timesteps when producing forecasts.

Dropout regularization (rate $p = 0.1$) is applied to both attention and feed-forward sublayers, and residual connections with layer normalization stabilize training. The encoder output corresponding to the last input token ($S - 1$) is passed through a linear readout layer to produce h future predictions:

$$\hat{\mathbf{y}}_{t+1:t+h} = \mathbf{W}_{\text{out}} \mathbf{z}_{t,S-1} + \mathbf{b}.$$

The loss function is the mean squared error (MSE) between normalized predicted and true values. Normalization statistics (mean and standard deviation) are computed exclusively on the training split to avoid data leakage, and the final predictions are denormalized for reporting in original price units.

For classification, the regression head is replaced with a single-output linear layer followed by a sigmoid activation:

$$\hat{p}_t = \sigma(W_{\text{cls}} \mathbf{z}_{t,S-1} + b),$$

and binary cross-entropy loss is used as the optimization objective. The Transformer models are trained using the Adam optimizer with learning rate 10^{-3} , weight decay 10^{-4} , and gradient clipping ($\|\nabla\| \leq 1$). Early stopping is applied.

Overall, the Transformer captures long-range dependencies across multiple temporal and feature dimensions, whereas XGBoost focuses on local, tabular correlations. By maintaining same windowing, normalization and task hyperparameters across both architectures, a fair comparison can be made between traditional tree ensembles and modern attention-based sequence models.

IV. EXPERIMENTS AND RESULTS

A. Dataset and Configuration

All experiments were performed on the ETH/USDT cryptocurrency pair using one-minute OHLCV (Open, High, Low, Close, Volume) data collected from the Binance public archive. To maintain consistency and temporal ordering, only the first month of January 2024 was used. The data were divided chronologically into training (70%), validation (15%), and test (15%) splits, ensuring no temporal leakage across sets.

For both forecasting and classification tasks, a sliding window approach was applied. Each model receives an input window of length $L_{\text{in}} = 150$ past timesteps and predicts the next h timesteps ahead. In forecasting, $h = 30$, while for directional classification $h = 1$. The stride between consecutive windows is 1 for forecasting and 8 for classification.

The XGBoost models use 400 estimators with a maximum tree depth of 6, subsampling and column sampling ratios of 0.8, and a histogram-based grower for efficiency. All experiments fix the random seed to ensure reproducibility.

The Transformer models employ an encoder-only architecture with $d_{\text{model}} = 128$, $n_{\text{heads}} = 8$, and 4 layers with a dropout rate of 0.1. Training is performed with the Adam optimizer using a learning rate of 5×10^{-4} for forecasting and 10^{-3} for classification, over 30 epochs and a batch size of 256, with early stopping after 15 epochs of no improvement. These configurations, summarized in the YAML files, provide a consistent experimental framework for both regression and classification settings.

B. Forecasting Results

The forecasting task is evaluated using two metrics: Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE). MAE measures the average magnitude of prediction errors without considering their direction, RMSE penalizes larger deviations more strongly.

TABLE I
MULTI-HORIZON FORECASTING RESULTS (LOWER IS BETTER).

Method	MAE	RMSE
XGBoost (Close-only)	4.376	6.364
XGBoost (OHLCV)	4.317	6.217
XGBoost (OHLCV + FE)	4.359	6.245
Transformer (Close-only)	3.778	5.364
Transformer (OHLCV)	3.912	5.514

Table I presents the denormalized MAE and RMSE values for all five model configurations. The Transformer (Close-only) achieves the lowest error across both metrics, confirming its superior ability to model temporal dependencies in the price series. XGBoost models improve moderately when using the full OHLCV feature set compared to Close-only inputs, while the addition of engineered indicators (returns, volatility, RSI, MACD, volume rate) yields limited benefit and slightly higher variance. This suggests that the features used are partially redundant when deep sequential encoders are employed.

Figure 1 illustrates an example forecast window for the Transformer model on the test set. The predicted h -step trajectory follows the ground truth closely, capturing local trends and inflection points while maintaining temporal smoothness.

C. Directional Classification Results

The classification task aims to predict whether the closing price after horizon h increases or decreases relative to the current value. Performance is assessed using Accuracy (ACC) and F1-score, along with the Area Under the ROC Curve (AUC). Accuracy captures the overall correctness of predictions, while F1 balances Precision and Recall, providing a robust measure under class imbalance. AUC, being threshold-free, quantifies the global ranking ability of the model.

TABLE II
DIRECTIONAL CLASSIFICATION AFTER HORIZON h (HIGHER IS BETTER).

Method	ACC	F1	AUC
XGBoost (Close-only)	0.481	0.492	0.490
XGBoost (OHLCV)	0.528	0.525	0.507
XGBoost (OHLCV + FE)	0.506	0.491	0.502
Transformer (Close-only)	0.493	0.660	0.512
Transformer (OHLCV)	0.493	0.660	0.509

Table II summarizes the classification metrics for all models. XGBoost with OHLCV inputs achieves the highest Accuracy (0.528), reflecting its strong discriminative ability on tabular features. However, the Transformer models reach substantially higher F1-scores, showing better recall-precision balance on upward trend detection. Although the absolute AUC values hover around 0.5 due to the symmetric up/down distribution, both architectures capture meaningful temporal patterns that outperform random baselines.

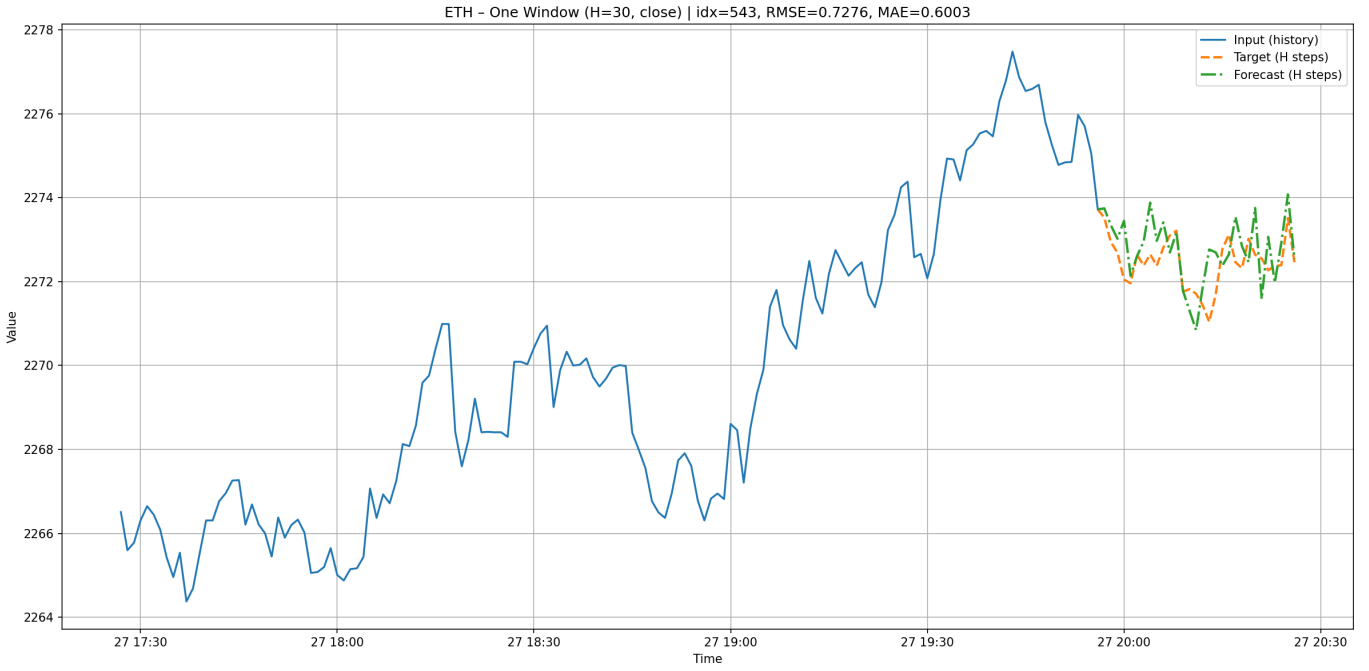


Fig. 1. Example forecast window using the Transformer (Close-only) model. Predicted h -step prices align closely with the observed values.

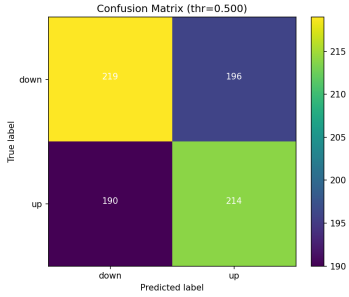


Fig. 2. Confusion matrix for XGBoost (OHLCV) predictions on the test set.

Figure 2 displays the confusion matrix for the XGBoost (OHLCV) classifier on the test set. The model correctly identifies most upward movements but tends to overpredict the majority class during minor reversals.

D. Efficiency and Model Size

Beyond predictive accuracy, we evaluate each model’s computational efficiency and scalability. XGBoost models train faster and on a CPU and require minimal memory, making them ideal for lightweight or real-time deployment. In contrast, Transformer models demand GPU acceleration and are frequently more heavy per epoch depending on their depth and embedding dimension. The largest configuration (Transformer with OHLCV inputs) contains roughly 2.1 million parameters, and occupies a memory of 8.32 mb.

While Transformers are computationally heavier, their superior accuracy and ability to model long temporal contexts justify the additional cost. In settings prioritizing speed, XG-

Boost with engineered features remains a practical alternative, balancing efficiency and predictive power.

Overall, the Transformer framework demonstrates clear advantages in continuous forecasting accuracy, while XGBoost remains competitive for directional classification and low-latency use cases.

V. DISCUSSION AND INSIGHTS

The attention-based architecture provides interpretability by revealing which past timesteps the model prioritizes when generating forecasts. The results confirm that the Transformer learns to focus on recent volatility bursts and regime transitions, highlighting its advantage in modeling temporal dependencies over long contexts. In contrast, XGBoost relies on static features, which limits its ability to exploit temporal relationships. Nevertheless, on shorter sequences or when feature engineering effectively summarizes past dynamics, XGBoost remains competitive, offering a strong baseline with excellent computational efficiency.

VI. CONCLUSION AND FUTURE WORK

In summary, the Transformer achieves lower forecasting errors and more balanced directional classification compared to XGBoost. XGBoost remains preferable for real-time inference due to its lightweight nature. Future work will explore multimodal extensions that incorporate market sentiment and volatility indicators.

VII. REPRODUCIBILITY

All code is publicly available in the following GitHub repository: <https://github.com/ramirovaldesjara/Stock-Forecasting-Transformer>. The repository contains the full pipeline, YAML

configurations for both forecasting and classification, and the scripts used for training and evaluation.

APPENDIX

```
Epoch 001 | Train: loss=0.9630 ACC=0.490 F1=0.510 AUC=0.492 | Val: loss=0.6965 ACC=0.455 F1=0.503 AUC=0.484
Epoch 002 | Train: loss=0.7120 ACC=0.494 F1=0.510 AUC=0.495 | Val: loss=0.7549 ACC=0.468 F1=0.637 AUC=0.502
Epoch 003 | Train: loss=0.7072 ACC=0.501 F1=0.464 AUC=0.501 | Val: loss=0.7642 ACC=0.468 F1=0.637 AUC=0.507
Epoch 004 | Train: loss=0.7044 ACC=0.495 F1=0.531 AUC=0.497 | Val: loss=0.6914 ACC=0.532 F1=0.000 AUC=0.492
Epoch 005 | Train: loss=0.6972 ACC=0.496 F1=0.474 AUC=0.499 | Val: loss=0.6976 ACC=0.468 F1=0.637 AUC=0.490
Epoch 006 | Train: loss=0.6967 ACC=0.501 F1=0.482 AUC=0.502 | Val: loss=0.7354 ACC=0.468 F1=0.637 AUC=0.518
Epoch 007 | Train: loss=0.6966 ACC=0.519 F1=0.488 AUC=0.515 | Val: loss=0.6923 ACC=0.523 F1=0.269 AUC=0.494
Epoch 008 | Train: loss=0.6985 ACC=0.497 F1=0.506 AUC=0.500 | Val: loss=0.6912 ACC=0.532 F1=0.000 AUC=0.507
Epoch 009 | Train: loss=0.6999 ACC=0.491 F1=0.457 AUC=0.498 | Val: loss=0.7012 ACC=0.468 F1=0.637 AUC=0.504
Epoch 010 | Train: loss=0.6949 ACC=0.493 F1=0.548 AUC=0.497 | Val: loss=0.6912 ACC=0.532 F1=0.000 AUC=0.499
Epoch 011 | Train: loss=0.6936 ACC=0.499 F1=0.508 AUC=0.507 | Val: loss=0.6909 ACC=0.537 F1=0.050 AUC=0.495
Epoch 012 | Train: loss=0.6953 ACC=0.510 F1=0.490 AUC=0.505 | Val: loss=0.6942 ACC=0.463 F1=0.577 AUC=0.501
Epoch 013 | Train: loss=0.6955 ACC=0.498 F1=0.551 AUC=0.503 | Val: loss=0.6974 ACC=0.532 F1=0.000 AUC=0.484
Epoch 014 | Train: loss=0.6971 ACC=0.500 F1=0.425 AUC=0.497 | Val: loss=0.7306 ACC=0.468 F1=0.637 AUC=0.497
Epoch 015 | Train: loss=0.6933 ACC=0.503 F1=0.461 AUC=0.513 | Val: loss=0.7001 ACC=0.468 F1=0.637 AUC=0.493
Epoch 016 | Train: loss=0.6941 ACC=0.508 F1=0.520 AUC=0.506 | Val: loss=0.6982 ACC=0.468 F1=0.637 AUC=0.476
Epoch 017 | Train: loss=0.6931 ACC=0.513 F1=0.544 AUC=0.513 | Val: loss=0.6977 ACC=0.468 F1=0.635 AUC=0.482
Epoch 018 | Train: loss=0.6945 ACC=0.522 F1=0.528 AUC=0.521 | Val: loss=0.6911 ACC=0.534 F1=0.005 AUC=0.494
Epoch 019 | Train: loss=0.6937 ACC=0.504 F1=0.504 AUC=0.512 | Val: loss=0.7015 ACC=0.468 F1=0.637 AUC=0.472
Epoch 020 | Train: loss=0.6937 ACC=0.508 F1=0.515 AUC=0.516 | Val: loss=0.6988 ACC=0.470 F1=0.638 AUC=0.477
Epoch 021 | Train: loss=0.6918 ACC=0.510 F1=0.519 AUC=0.519 | Val: loss=0.6937 ACC=0.498 F1=0.262 AUC=0.478
Early stopping.
Saved confusion matrix → C:\Users\rjv71\PycharmProjects\Stock_Forecasting_Transformer\transformer\transformer_cls_ohlcv_ETH_150to1\transformer_cls_ohlcv_confusion_matrix.png

Classification metrics (ACC/F1/AUC):
Train: {'loss': 0.696573271672912, 'ACC': 0.518775720164609, 'F1': 0.4883784522832923, 'AUC': 0.5150511955778241}
Valid: {'loss': 0.6923235876074059, 'ACC': 0.5225885225885226, 'F1': 0.2691588785046729, 'AUC': 0.49353845785325895}
Test : {'ACC': 0.4932844932844933, 'F1': 0.660670482420278, 'AUC': 0.5096325897650006}

Process finished with exit code 0
```

Fig. 3. Screenshot of a complete training and evaluation run for the ETH/USDT classification experiment.